

Homework #1

Analysis of Algorithms

Illya Starikov

Due Date: September 1st, 2017

Question #1

- (a) 4.54 ± 0.05 billion years (https://en.wikipedia.org/wiki/Age_of_the_Earth)
- (b) 5 billion years (http://earthguide.ucsd.edu/virtualmuseum/ita/05_3.shtml)
- (c) 13.6 billion years $\pm$ 800 million years (<https://www.space.com/263-milky-age-narrowed.html>)
- (d) 13.82 billion years (http://www.slate.com/blogs/bad_astronomy/2013/03/21/age_of_the_universe_planck_results_show_universe_is_13_82_billion_years.html)
- (e) 7.79 billion years (<https://www.livescience.com/39775-how-long-can-earth-support-life.html>)
- (f) 3.25 billion years. Guaranteed existential risks, human prevention not possible.
 - Asteroid impact
 - Extraterrestrial invasion (i.e. , aliens taking over the world)
 - Geomagnetic reversal of the North and South pole

Prevention possible.

- Artificial super intelligence enslavement of the man race. Steps are unclear, besides keeping all experiments in a Faraday cage.
- Biotechnology engineering. A bio-engineered virus or disease that gets loose can cause a global pandemic. Steps to prevent are clear.
- Nuclear holocaust. Steps to prevent are clear.

Extinction prevention matters are to become a space-ferrying species, attempt some sort of peace treaty amongst all nations, and to try to prevent global pandemics.

(https://en.wikipedia.org/wiki/Global_catastrophic_risk#Asteroid_impact)

- (g) 10-12 billion years. The sun will become a Red Giant, no longer being able to provide fuel. Will consume several planets along the way. (<https://www.forbes.com/sites/startswithabang/2017/01/27/how-our-solar-system-will-end-in-the-far-future/#20ea714f4e5d>)

Eventually, about 5–7 billion years down the line, we'll run out of nuclear fuel in the Sun's core, which will cause our parent star to become a Red Giant, engulfing Mercury and Venus in the process. Due to the particulars of stellar evolution, the Earth/Moon system will probably be pushed outwards, and be spared the fiery fate of our inner neighbors.

- (h) Either the universe will stop contracting, reach an influx in the gravitational pull on all cosmic bodies, and come back to a singularity; or, heat death, where entropy reaches its max, and all energy is evenly dissipated throughout the universe, freezing

everything. The first theory has 1-100 trillion years, the second has no estimate.
(<http://www.bbc.com/earth/story/20150602-how-will-the-universe-end>)

- (i) 584.6 billion years. Between 0.585%–58.5%.

Question #2

- (a) A standard, 12pt L^AT_EX document can fit roughly 39 lines of text.

$$\frac{9 \times 10^8}{39} \approx 2.31 \times 10^8 \text{ sheets}$$

- (b) 1500 sheets is approximately \$14.99, before tax. (https://www.amazon.com/Georgia-Pacific-Spectrum-Standard-Multipurpose-998606/dp/B00BB5DJU6/ref=sr_1_2?s=office-products&ie=UTF8&qid=1503934559&sr=1-2&keywords=printer+paper)

$$(2.31 \times 10^8 \text{ sheets}) \times \frac{\$14.99}{1500 \text{ sheets}} \approx \$2.31 \times 10^6$$

Assuming a standard filing cabinet can store roughly 15 reams, with 100 sheets in a ream,

$$2.31 \times 10^8 \text{ sheets} \times \frac{1 \text{ ream}}{100 \text{ sheets}} \times \frac{1 \text{ cabinet}}{15 \text{ reams}} = 154\,000$$

- (c) The monks expected the project to take *15,000 years* by hand, and *3 months* by modern technology. Assuming an average human lifespan of 79 years, approximately 50 years could be used writing.

$$\frac{15\,000 \text{ years}}{50 \text{ years}} = 300 \text{ people}$$

- (d) The time allocated to the computer was *3 months*. To print all 3 billion, it would have to print approximately 1120.07 names per second.
- (e) I enjoyed the story, and the first thing I did was a mental check to see if the numbers added up. Naturally, they did not, but that did not detract from the story.

Question #3

```
1 def alternating_difference(numbers):
2     if not numbers:
3         return 0
4
5     return numbers[0] - alternating_difference(numbers[1:])
```

Question #4

```
1 def f53q(n):
2     if n < 8:
3         return ValueError('Value_less_than_8')
4     elif n % 5 == 0:
5         return (0, n // 5)
6     elif n % 5 == 1:
7         return (2, (n - 5) // 5)
8     elif n % 5 == 2:
9         return (4, (n - 12) // 5)
10    elif n % 5 == 3:
11        return (1, n // 5)
12    else:
13        return (3, (n - 9) // 5)
```

Question #5

$$\begin{aligned}f(1) &= 91 \\f(-6) &= 91 \\f(200) &= 190 \\f(27) &= 91\end{aligned}$$

A more appropriate, non-recursive function would be as follows:

```
1 def f(n):
2     if n < 101:
3         return 91
4     else:
5         return n - 10
```

This will always produce the same output. Let us write $f(n)$ as a piecewise-defined function.

$$f(x) = \begin{cases} n - 10 & 100 < n < \infty \\ f(f(n + 11)) & -\infty < n \leq 100 \end{cases}$$

We see that, for values less than 100, n will grow until it hits the first case. For values $90 \dots 200$, we see that n will oscillate until it reaches 101, upon which one final $f(x)$ will be applied.

A more simple explanation would be to insert $n - 10$ into $f(f(n + 11))$ to see that $f(x) = f(f(n + 1))$ until it reaches 101, 1 past the bounds, and gets $n - 10$ applied to it one more time.

Question #6

The function is copy and pasted from implementation.

```
1 def A(x, y):
2     if x == 0:
3         return y + 1
4     elif y == 0:
5         return A(x - 1, 1)
6     else:
7         return A(x - 1, A(x, y - 1))
```

The maximum value of my system (Late-2013 MacBook Pro) is roughly $A(3, 5)$. For $A(2, 2)$, there should be 42 438 function calls.

Question #7

```
1 def gcd2(a, b):
2     if a == 0:
3         return (b, 0, 1)
4     else:
5         g, s, t = gcd2(b % a, a)
6         return [g, t - (b // a) * s, s]
```

Question #8

```
1  # defined as list_ to not stomp out the typename _list_
2  def super_reverse(list_):
3      if len(list_) < 2:
4          return list_
5      else:
6          last_element = list_[-1]
7          first_element = list_[0]
8
9          # if is a list or a tuple, sort that as well
10         if isinstance(first_element, list):
11             first_element = super_reverse(first_element)
12         if isinstance(last_element, list):
13             last_element = super_reverse(last_element)
14
15         return [last_element] + super_reverse(list_[1:-1]) + [
            first_element]
```

Question #9

```
1 def anagram(string_):
2     if string_ == "":
3         return [""]
4
5     result = []
6
7     for partial_anagram in anagram(string_[1:]):
8         for position in range(len(partial_anagram) + 1) :
9             right_side = partial_anagram[position:]
10            left_side = partial_anagram[:position]
11            original_element = string_[0]
12
13            result += [left_side + original_element + right_side]
14
15     return result
```

Table 1: Results (units in milliseconds)

String Size	One Recursive Call	n Recursive Calls	Absolute Difference	Percent Difference
1	0.006	0.004	-0.002	-56.250%
2	0.009	0.009	-0.000	-2.703%
3	0.015	0.026	0.011	42.202%
4	0.036	0.104	0.068	65.367%
5	0.139	0.528	0.389	73.668%
6	0.672	3.124	2.452	78.486%
7	5.312	25.050	19.738	78.794%
8	39.671	204.850	165.179	80.634%
9	332.021	1936.445	1604.424	82.854%
10	3471.336	21813.828	18342.492	84.087%
11	43223.737	267635.574	224411.837	83.850%